Stochastics and Machine Learning
Spring 2024

ETH Zürich
Cheridito, Cotrini and Streich

Problem Set 9: Neural Networks
(May 10 - May 23, 18:00)

Problem 1 (Equivalence of two layers with a linear activation to a single layer)

Let us have two layers of neurons with a linear activation function.

Particularly, let the input to the first layer consist of L real numbers $x_1, x_2, ..., x_L$. Let the first layer consist of M neurons. Since the neurons have a linear activation, the output of i-th neuron is equal to

$$z_i = \sum_{j=1}^{L} v_{ij} x_j, \tag{1}$$

where v_{ij} is the weight for the j-th input in the i-th neuron of the first layer.

Let the second layer consist of N neurons. The output of the i-th neuron of the second layer is equal to

$$y_i = \sum_{j=1}^{M} w_{ij} z_j, \tag{2}$$

where w_{ij} is the weight for the j-th input in the i-th neuron of the second layer.

Show that these two layers are equivalent to a single layer with a linear activation function.

Problem 2 (Convolutional Neural Networks)

1. List at least two reasons why we would typically choose a convolutional layer over a fully connected layer when processing image data.
2. Consider the following 1D signal: $[5, 0, -3, 1, 4]$. After convolution with a filter of length=3, padding=0, and stride=1, the following sequence is obtained: $[15, 14, -1]$. What is the filter?

Problem 3 (CNNs on CIFAR10)

Please check the corresponding notebook for more details.

Neural network parameters

Consider the CNN class defined in the lecture and also included in `cnn.py` for convenience.

1. Compute the number of trainable parameters. Assume all parameters are trainable.
2. Compute the receptive field of each output after each of the pooling layers.
3. Repeat questions 1. and 2. for the case where a third convolution with 128 filters (using same kernel size as the others) and pooling layer (same as the others) are added.

```python
import torch.nn as nn
import torch.functional as F

class CNNModel(nn.Module):
    def __init__(self):
        super(CNNModel, self).__init__()
        num_classes = 10
        # First convolutional layer with 32 filters
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)

        # Second convolutional layer with 64 filters
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)

        # Max pooling layer
        self.pool = nn.MaxPool2d(2, 2)

        # Fully connected layers
        self.fc1 = nn.Linear(64 * 7 * 7, 128)  # 64 filters, 7x7 spatial size after max pooling
        self.fc2 = nn.Linear(128, num_classes)  # Output layer with 10 classes

    def forward(self, x):
        # Forward pass through the network
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 64 * 7 * 7)
        x = self.fc2(self.fc1(x))
        return x
```